

AUGUST-SEPTEMBER 2026

LEANNA FOWLER

Telco Customer Churn

Analyzing customer churn

Leanna Fowler

The purpose of this project is to build skills in data-driven decisions.

I decided to start simple, by analyzing churn rates and assessing what might help in the future.

What's in this slide deck

Purpose: to go over my thoughts, planning, execution, and results.

1. Dataset choice
2. Loading the data
3. Performing EDA
4. Investigating the features
5. Training a model
6. Explaining future steps
7. Limitations
8. Key learnings and takeaways

The Dataset: Telco (Telecommunications Company) Churn

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

Context:

- This dataset is a sample data set from IBM, uploaded to Kaggle by blastchar

The dataset:

- Contains 7043 rows (customers) and 21 columns (features)
- Each row represents a singular customer, and each column is a feature
- These are customers who have left within the last month (indicated in the “Churn” column)
- Services that each customer has signed up for - phone, multiple lines, internet, etc
- Customer accounting information
- Demographic information

Step 1: Load

After looking at the dataset, I thought it was way too big, so I decided to perform some simple assessments to see what I was dealing with - before I begin loading the data for analysis.

I used Jupyter notebook (.ipynb) to look at any null values, duplicate values, and data types. Found that there are no duplicates, no null values, but some data types were not suitable for analysis. One of such is “TotalCharges”, where I changed it to ‘float64’ from ‘object’. Another one I adjusted was the ‘Churn’ column, so that the yes/no is defined as 1/0.

After cleaning the data, I loaded it as a dataframe in the ‘load_clean’ function.

```
1 def clean(df: pd.DataFrame) -> pd.DataFrame:
2     df = df.copy()
3     # clean the db
4     # found "TotalCharges" as "Object" -> change to float64
5     # blank TotalCharges are all tenure == 0 (brand-new customers,
6     # not yet billed)
7     df['TotalCharges'] = pd.to_numeric(df['TotalCharges'],
8     errors='coerce').fillna(0)
9
10    # Changing "Churn" to be 0 or 1 (Yes/No)
11    df['Churn'] = (df['Churn'] == "Yes").astype(int)
12
13    # customerID is not a feature, so drop it
14    df = df.drop(columns=["customerID"])
15    return df
```

in `data\_loader.py`

Step 2: EDA

Once I completed the cleaning of the data, I began performing EDA, looking at churn rates by segment, and saving charts as images for viewing.

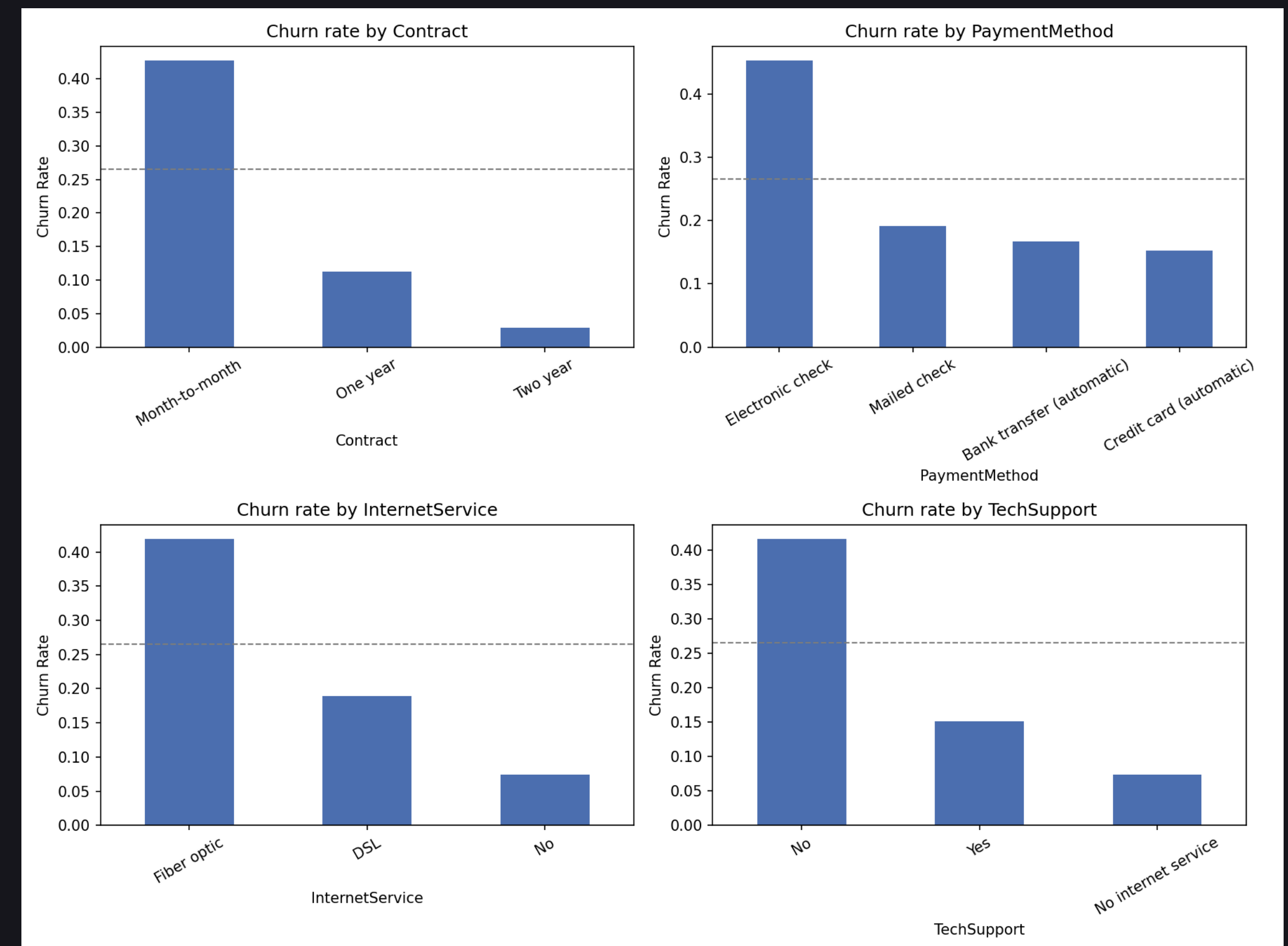
I identified the main drivers of the dataset and created a list of segments:

```
1 segments = ["Contract", "PaymentMethod", "InternetService", "TechSupport"]
```

Each segment is looked at and graphed as a bar chart.

The output is *churn_by_segment.png*.

From the chart, what we can take away from this is that the churn rates are quite high in all segments, **where the highest churn rate is around 45% - not ideal!**



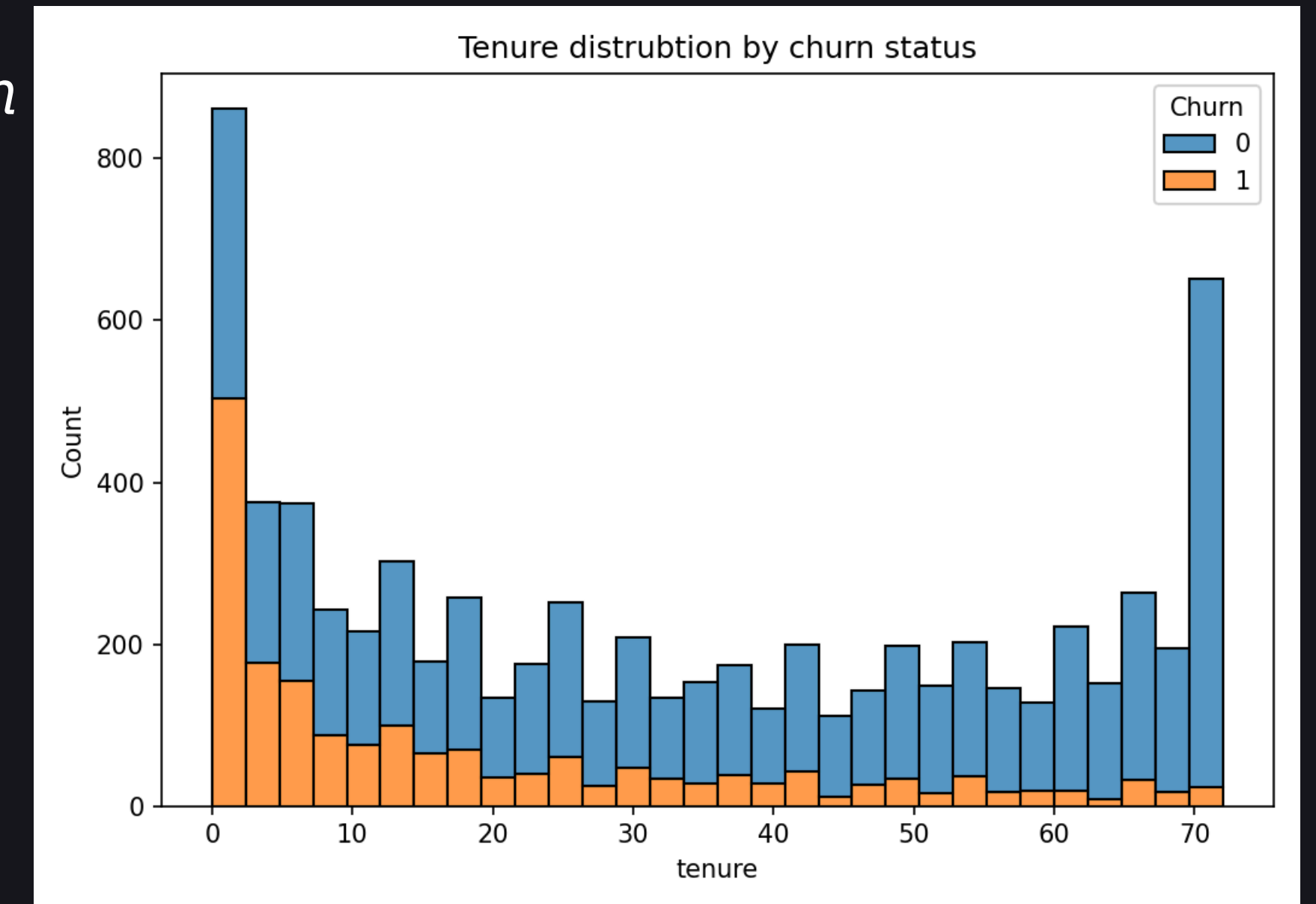
PERFORMING EDA

Quick note: 0 = No Churn, 1 = Churn

Step 2: EDA

I also looked at tenure, by creating a histogram: *tenure_by_churn.png*

```
1 sns.histplot(data=df, x="tenure", hue="Churn", bins=30, multiple="stack", ax=ax2)
```



What do we see from our stacked histogram?

- The very first bin is the tallest bar in the whole chart, and it's also the only bar where the orange (churned) segment is roughly half the bar rather than a thin sliver. That's the huge spike at the left edge
- Moving to the right of the chart, total customers drop off quickly through the first (around) 20 month, and the orange churned share shrinks with it

What can we take away from this?

- New customers are both the largest group and by far the riskiest. The first few months carry a churn rate way above every other tenure range, so churn risk is heavily front loaded rather than spread evenly
- Long tenured customers rarely churn
- **Therefore - retention effort is most valuable in a narrow early window, since that's where most of the churned customers are concentrated**

Step 3: Looking at Features

For features, I first load and clean in `data_loader.py`, then derive features in `feature_engineering.py`.

There are around 7000 customers, 19 raw predictors (contract, payment, tenure, charges, 9 service flags), and the Churn label.

Cleaning: I made 2 decisions.

1. TotalCharges blank - for these, I made sure that all are brand-new customers, where tenure=0, so we set to 0:

```
1 df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce').fillna(0)
```

2. Dropped customerID. This is an identifier, not a feature/signal

Step 3: Looking at Features

Engineering the features: I did add 4 from domain reasoning.

- A couple of the engineered features overlap with raw ones (tenure_bucket with tenure, is_electronic_check with its own dummy).
- XGBoost handles the redundancy; The next iteration would prune them so SHAP attribution isn't split across duplicates.

Feature	Definition	Why it should predict Churn
tenure_bucket	tenure binned: 0-6mo ... 4yr+	churn risk concentrates in the first year; lets the model treat "new" as a category
addon_count	# of add-on services subscribed (0-6)	bundle depth = switching cost = stickiness
charge_ratio	TotalCharges ÷ MonthlyCharges	flags accounts spending less than their tenure implies
is_electronic_check	payment method flag	highest-churn segment in EDA

Final matrix is around 30+ columns

Step 4: Model Training

I chose to train two (2) models - Logistic Regression (baseline) and XGBoost

Why these two? I chose them as a pair, a simple transparent baseline and a strong challenger.

Logistic Regression (baseline)

- It's interpretable. Each coefficient is “holding everything else equal, this multiplies the odds of churn by X”.
- The bar every model has to clear. Because if a more complex model can't beat a well-tuned logistic regression, the complexity isn't earning its keeping. So should ship the simple one!
- It's fast, stable, and only few things to get wrong
- Lastly, it fits the shape of this problem. Churn here is driven mostly by main effects like contract, tenure, and payment method, rather than complex interactions, which is exactly what a linear model is best at

Step 4: Model Training

XGBoost (challenger)

- It captures non-linearity and interactions automatically. It can learn terms without me hand-coding that term. A linear model only sees each feature in isolation unless you build the interaction yourself
- XGBoost is a default for tabular data, since it's a tree-ensemble algorithm
- It's robust to messy imports, as no feature scaling is needed, handles missing values natively, deals with class imbalance via the `scale_pos_weight`
- Also pairs naturally with SHAP. There is an algorithm called TreeSHAP, designed for tree-based models like XGBoost. This integration allows us to calculate the exact Shapley values for complex tree ensembles in polynomial time (not exponential time). `TreeExplainer` computes exact SHAP values fast for tree models, so the explainability step is cheap!

Step 4: Model Training

Why train the models?

1. To really help turn “who might churn?” into a ranked list

The business can't act on all 7,043 customers because a retention campaign might have a fixed budget and can only contact a limited number of accounts. Training a model produces a churn-risk for every customer. This is why the headline metric is precision-at-top-20%, not accuracy. The question we are asking is “if we call the 20% the model flags as riskiest, what fraction are real churners?” - I answered this question at it's around 66%, versus if I picked at random, 27%

2. To help identify what's driving churn, not just predict it

A trained model + SHAP tells us which factors push a customer's risk up or down, and by how much. This is what feeds the retention recommendations - contract type, early tenure, payment method, and unsupported fiber all surfaced as top drivers because the model weighted them heavily, and SHAP made that legible. The model is an instrument for understanding the problem!

3. To test whether complexity is worth it or not

Training a simple, interpretable baseline (logistic regression) alongside a stronger model (XGBoost) answers a practical question... does the business need a black box, or is a transparent model good enough? Here, the baseline matched XGBoost, so the answer is that the simple model is sufficient!

Step 4: Model Training

I am training two (2) models - Logistic Regression (baseline) and XGBoost

How the training works:

```
model.fit(X_train, y_train)
```

- I am handing the algorithm the feature matrix 'X_train' (contract type, tenure, charges, ...) and the known answers 'y_train'
 - The algorithm adjusts its internal parameters until its predictions on those rows match the known answers as closely as possible
 - Then calling the following (code snippet below) on customers it never saw, and check whether it got those right too
- ```
predict_proba(X_test)
```

## Step 4: Model Training

### Model 1: Logistic Regression

```
def train_baseline(X_train, y_train)
```

The baseline: this is a deliberately simple reference point

- a linear model: it multiplies each feature by a learned weight, sums them, and squashes the results through an S-curve into a probability between 0 and 1. The weight sign tells direction (month-to-month contract → weight pushes risk up), magnitude tells the strength
- this is fast, but can only draw a straight-line boundary between “churn” and “stay”

```
1 def train_baseline(X_train, y_train):
2 scaler = StandardScaler()
3 X_train_scaled = scaler.fit_transform(X_train)
4 # class_weight="balanced" up-weights the minority (churn) class so the model doesn't just learn to predict "no churn" for everyone
5 model = LogisticRegression(max_iter=1000, class_weight="balanced")
6 model.fit(X_train_scaled, y_train)
7 return model, scaler
```

## Step 4: Model Training

### Model 2: XGBoost

*def train\_xgb(X\_train, y\_train)*

What is XGBoost?

- Gradient-boosted decision trees - it builds many shallow decision trees in a sequence (here it's 300, each with depth 4). Each new tree is trained to correct the errors the previous trees are still making. The final prediction is the sum of all trees

A single depth-4 tree is a stack of yes or no questions:

- "contract = month-to-month? → tenure < 6 months? → ...")

Boosting hundreds of them together captures nonlinear patterns and feature interactions that logistic regression can't, which is why it usually wins of ROC-AUC (the area under the receiver operating characteristic curve) here

```
1 def train_xgb(X_train, y_train):
2 neg, pos = (y_train == 0).sum(), (y_train == 1).sum()
3 model = xgb.XGBClassifier(
4 n_estimators=300, # number of boosting rounds (trees)
5 max_depth=4, # shallow trees -> less overfitting
6 learning_rate=0.05, # small steps
7 scale_pos_weight=neg / pos,
8 eval_metric="logloss",
9 random_state=42,
10)
11 # Trees don't need scaling, so raw X_train goes straight in.
12 model.fit(X_train, y_train)
13 return model
```



# Step 4: Model Training

A rundown of the other functions in `train_model.py`

| Function                    | Category   | What it does                                                                                     | Relation to model training                                                                                                                                   |
|-----------------------------|------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>split_data</code>     | data prep  | Splits Churn off as the target y, keeps the rest as X, then holds out 20% of rows as a test set. | Feeds training — <code>.fit</code> only ever sees the 80% <code>X_train/y_train</code> . The 20% <code>X_test</code> is quarantined so evaluation is honest. |
| <code>precision_at_k</code> | evaluation | given true labels + predicted risk scores, computes precision within the top-k% riskiest         | This function is not involved in training. it runs after, on predictions                                                                                     |

# Step 4: Model Training

A rundown of the other functions in `train_model.py`

| Function              | Category               | What it does                                                                                                                   | Relation to model training                                                                           |
|-----------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>evaluate</code> | evaluate/<br>reporting | Runs ROC-AUC, <code>precision_at_k</code> , and <code>classification_report</code> → prints them; returns a metrics dictionary | After training. Never touches model internals; it just compares predictions to <code>y_test</code> . |

## Step 5: Future Steps *What the analysis is telling me:*

Churn is not evenly spread; it's four identifiable pockets.

- Contract is the dominant lever
  - Month-to-month customers churn at 42.7%; one-year at 11.3%; two-year at 2.8%. Roughly 15x difference. 55% of the base is month-to-month, so the single biggest risk group is also the largest
- The first six months decide everything
  - 52.9% of customers with 0–6 months tenure churn, and that one-fifth of the base accounts for over 40% of all churn. After four years, churn is under 10%. Retention is won or lost early
- Electronic-check payers are a red flag. 45.3% churn --> more than double every other payment method
  - Likely a marker for low-commitment customers, like who never set up autopay, as much as the payment method itself
- Fiber without tech support is a distinct failure mode
  - 49.4% churn vs. 7.4% for customers with no internet service and 15.2% for those who have tech support attached



## Step 5: Future Steps

### **What Telco can do**

1. Structure a 90-day onboarding program --> welcome check-in, proactive support outreach
  - a. By the time a risk score flags them, they're already in the danger window. Get ahead of it
  - b. Here, will be targeting all new signups
  
2. Discounted annual-contract upgrade offer
  - a. Would be useful because moving even 10% to annual terms meaningfully cuts expected churn
  - b. Here, will be targeting month-to-month customers
  
3. Bundling a free 2-3 month tech-support trial into fiber plans at signup
  - a. Attacks the highest-churn service tier with the add-on most associated with staying
  - b. Here, will be focusing on fiber customers without tech support



## Step 5: Future Steps

Onboarding and the fiber/tech-support bundle are the surest options for the future because they change the actual customer experience. The contract offer is high-impact but will need to confirm that I'm retaining customers rather than discounting ones who's have stayed.

But, there is one gap before I can size any of this. The dataset has no customer lifetime value or margin figures, so every number above is a customer count, not a dollar return. Telco has that data internally, so pairing it with these segments turns this into a budgeted plan.

# Limitations

It's a single snapshot with no time dimension. Every row is one customer frozen in time, so you can't see when someone churned, how long they'd been a customer when they left, or what changed right before. This means every finding is a correlation, not causation.

There are no dollar figures. There's no customer lifetime value, revenue margin, or cost-to-serve in the dataset. So you can rank segments by churn rate and headcount, but you can't say which segment is worth the most to save, or whether a given retention offer pays for itself. Every "estimated impact" is a customer count standing in for a financial return.

The dataset is missing the behavioural signals that usually explain churn.

The model's edge is modest and lightly validated. Risk ranking is about 2.5x better than random (top 20% riskiest are around 66% real churners). It's also validated on a single random split of ~1,400 customers, so the performance numbers carry real uncertainty and shouldn't be quoted to three decimals.

# Key Learnings and Takeaways

I learned...

- Churn concentrates in a few structural segments (contract, early tenure, payment method, service tier) rather than being diffuse
- A turned-enough linear baseline can match gradient boosting when the dataset is small and the signal is low-order
- The hard part of this churn project (or any churn project) isn't the model, but it's separating the "this predicts churn" from the "changing this reduces churn"



# Key Learnings and Takeaways

I got hands on experience with:

- Leakage-aware preprocessing - ensuring transformation parameters are computed strictly on training data rather than the entire dataset
  - fitting `StandardScaler()` on train only and reusing it on test
- Class imbalance handling
  - As this imbalance can happen when one data group appears much more often than another data group. This can cause standard models to ignore the rare group and make poor predictions
  - `class_weight="balanced"` for the linear model
  - `scale_pos_weight = neg/pos` for XGBoost
- Gradient-boosted decision trees, an ensemble that builds many shallow trees in sequence, where each new tree tries to connect the errors the previous tree made. This is the “boosting” part! versus a random forest, where trees are built independently and averaged

# Key Learnings and Takeaways

This is the first time I've touched customer churn as a problem. I took the raw telecom data, cleaned it, explored churn rates by segment, built some derived features, and trained two models to predict churn (a logistic regression baseline and an XGBoost model) then used SHAP to explain what was driving the predictions. Along the way I learned what gradient boosting actually is, why class imbalance has to be handled deliberately, why you fit a scaler on training data only, and that precision-at-top-decile matters more than accuracy when a retention campaign has a limited budget.

From this project, the main takeaway is that modeling is the easy part. The hard part is separating “this predicts churn” from “changing this reduces churn”, and answering that needs a time dimension, dollar values, and a real experience. none of which a single-snapshot dataset can give you.

26 JUNE 2024

Thank you.